A Comparative Analysis of Apache Spark Streaming and Apache Flink for Real-Time Data Processing Performance

Iman Abadi Bin Mohd Nizwan
Faculty of Computing
University Technology Malaysia
Johor Bahru, Malaysia
imanabadi@graduate.utm.my

***Abstract*— The rapid growth of real-time data generated from applications such as Internet of Things (IoT), financial systems and online platforms. This has created a demand for efficient stream processing frameworks capable of delivering low-latency analytics. This paper focuses on Apache Spark Streaming as a widely used stream processing framework and examines its architecture, processing model and performance characteristics. This paper also explores on comparative analysis between Apache Spark Streaming against Apache Flink, with evaluation criteria including processing model, latency, throughput, scalability and fault tolerance. The findings shows that Apache Flink achieves better performance in terms of low-latency event-driven processing but Apache Spark Streaming remains highly effective for near real-time analytics due to its micro-batch model, strong fault tolerance and seamless integration within the Spark ecosystem.**

Keywords— Apache Spark Streaming, Apache Flink, Stream Processing; Real-Time Analytics

I. Introduction

The growth of real-time data from various types of applications and software has caused a need for an efficient real-time data processing solution. Applications and software such fraud detection systems, social media and Internet of Things (IoT) platforms produce high-velocity data that requires immediate insights and action. Traditional batch processing frameworks such as Apache Hadoop, although good for big data processing, lack the ability for low-latency data processing needed for real-time analysis. Hadoop is known to have high-latency and disk I/O overhead that significantly reduces performance time (Matteussi et al., 2022). To address these challenges, a stream processing framework offers better performance for stream analytics. Examples of streaming frameworks include Apache Spark Streaming, which uses a micro-batch model and Apache Flink, which uses a native streaming model. Both frameworks have their advantages and disadvantages in terms of performance, scalability and fault tolerance, for example. Selecting an appropriate streaming framework is crucial for an efficient streaming system that can optimize system performance and resource usage while meeting the requirements for a real-time processing system. This paper focuses on Apache Spark Streaming as a modern stream processing framework and compares it with Apache Flink based on key performance criteria, including processing model, latency, scalability and fault tolerance. The paper is structured into Apache Spark Streaming and architecture overview, a real-world case study, comparative analysis among stream processing frameworks and conclusion.

II. Apache Spark Streaming And Achitecture Overview

Apache Spark Streaming is a stream processing framework that offers rapid processing of large amounts of data through in-memory computations. This is preferrable compared to traditional map-reducing methods because it relies on disk operations, increasing I/O overhead, thus increasing latency. Spark Streaming processes data up to 100 times faster compared to Hadoop's map-reducing method (Ebada et al,. 2022).

Apache Spark Streaming uses micro-batching techniques, processing data in discrete intervals, and combining them to have continuous stream-like data which are then processed using Spark's underlying distributed computing engine which in turn enables it to have high throughput and fault tolerance (Fedorovych et al., 2024). The micro-batches are called Resilient Distributed Datasets (RDDs), a data abstraction method which are separate data blocks organized into small groups that corresponds to a batch data received at specific time interval which can be processed in parallel (Matteussi et al., 2022). The blocks are then sequenced overtime to form stream-like data. The architecture can be described in Fig. 1.

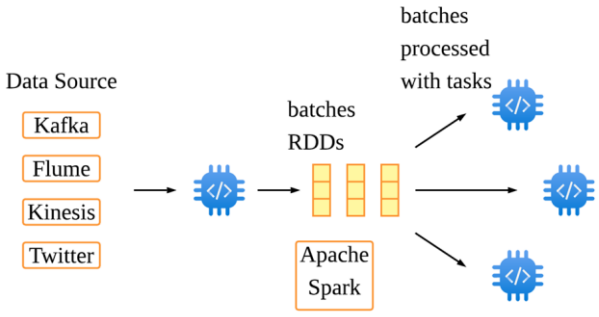


Fig 1. Apache Spark Streaming Architecture

Apache Spark Streaming also has excellent integration with other components in the Spark ecosystem such as MLlib. Apache Spark's MLlib is a Machine Learning (ML) library optimized for batch processing and micro-batch streaming (MEZATI & AOURIA, 2025). This integration provides a unified platform for complex analytics and machine learning which eliminates the need for multiple processing systems, simplifying the development of an end-to-end data pipeline.

Although Apache Spark Streaming is famous for its high throughput and scalability, it still lacks extremely low latency processing compared to other frameworks. In order to address this issue, Apache Spark Streaming introduced a more Structured Streaming which includes a new processing

model called Continuous Processing. This is done by using long-lived operators, keeping states across data batches. This reduces overhead that comes with micro-batch processing, having multiple task starts and stops (Fedorovych et al., 2024). This shows that Apache Spark Streaming gives option various application in stream analytics from using micro-batch processing for fault tolerance, high throughput and scalability or using Continuous Processing for improved latency for near real-time data processing.

III. Case Study: Apache Spark Streaming For Health Status Prediction

Ebada et al. (2022) proposed on using Apache Spark for streaming big data for health status prediction. The paper introduces an intelligent and scalable data architecture designed to monitor and predict health issues by leveraging on cloud computing and IoT devices. The primary goal is to create a system that interprets continuous medical data from wearable biosensors like heart rate, blood pressure, and body temperature trackers to detect health risks before they become critical. By combining past electronic health records (EHRs) with live streaming data, the system can deliver personalized suggestions and real-time alerts to patients and healthcare providers. Apache Spark is used for its capability to handle high-velocity healthcare data from IoT devices to extract medical data which will then be used by Apache Spark's machine learning library, MLlib to apply classification algorithms such as multiclass neural network and multiclass random forests. In summary, this study highlights the capabilities of Apache Spark Streaming in managing real-time medical data.

IV. Comparative Analysis on Stream Processing Frameworks

This section compares Apache Spark Streaming, a stream processing framework with another framework called Apache Flink. Apache Flink is an open-sourced framework that is known for distributed stream processing that is scalable, stateful and fault-tolerant (Vyas & Kaushik, 2025).

TABLE I. Comparison of Stream Processing Frameworks

Criteria	Apache Spark Streaming	Apache Flink
Processing Model	Micro-batch by default (Daksa & Kamala, 2025). Supports Continuous Streaming for true streaming (Fedorovych et al., 2024)	Native streaming (Daksa & Kamala, 2025)
Latency	Ranges from milliseconds up to seconds with more transactions (Le et al., 2022)	Milliseconds (Le et al., 2022)
Throughput	High, slows down under heavy load (Le et al., 2022)	High (Le et al., 2022)
Fault Tolerance	Checkpoint based on RDD (Le et al., 2022)	Checkpoint + Savepoint (Le et al., 2022)
Scalability	Linear with resources, presents in-memory issues under heavy load (Matteussi et al., 2022)	Linear with resources, better performing even under limited resources (Vyas & Kaushik, 2025)

A. Processing Model Comparison

The main difference between Apache Spark Streaming and Apache Flink is the processing models used which significantly effects system performance, latency and suitable use cases. Spark Streaming uses a micro-batching processing model where incoming data are separated into small batches and processed and s set time interval. Apache Flink on the other hand, is known for its true streaming approach, which processes individual events with sub-second latency and fine-grained state management (Daksa & Kamala, 2025).

Daksa & Kamala (2025) discussed that each paradigms has their own strength and specific use cases based on their literature review. Apache Spark Streaming's micro-batch approach ensures stronger fault tolerance but causes higher latency in term of processing speed. Flink on the other hand is more suited for real-time event processing as it offers low latency and higher processing consistency.

B. Performance Evaluation (Latency and Throughput)

A study by Le et al. (2022) showed that Apache Flink is better than Apache Spark Streaming in terms of performance in shuffle, stateful computation and windowed computation for both latency and throughput. The test was done using HiBench Streaming Benchmark using four types of test loads namely identity, repartition, stateful wordcount and fixwindow. In terms of throughput, results show that Apache Flink consistently outperforms Apache Spark Streaming. For identity and fixwindow test loads, both frameworks showed similar throughput up until 100,000 data records per second where Apache Sparks Streaming starts to drop. Apache Spark Streaming performed worse in repartition and wordcount test loads where throughput drops at 60,000 and 80,000 data records per second while Apache Flink maintains consistently high throughput. In terms of latency, Apache Flink consistently have latency in the millisecond range even at high data streams. Apache Spark Streaming on the other hand has latency in milliseconds range at low data streams but increases at higher workloads.

Another study by Daksa & Kamala (2025) showed contrasting results compared to the previous study. Apache Spark Streaming achieved lower latency for both ingestion rates (10 and 40 transactions pr seconds). Daksa & Kamala (2025) acknowledge that the findings differ from other literature as Apache Flink is always known to be better at low-latency processing. The paper found that Apache Spark Streaming's improved performance could be associated with internal optimization and consistent batch sizing that is aligned with workload characteristics. The article also suggests that Apache Flink is better suited for stateful processing and complicated event scenarios, which is demonstrated by Le et al. (2022), whereas Spark is recommended for predictable event structures and stable data rates.

C. Fault Tolerance and Scalability

Different stream processing models have different capabilities in terms of fault tolerance. According to Wang et al. (2022), a micro-batch processing model used by Apache Spark Streaming is near inherently fault-tolerant due to the underlying robust data structures and lineage information but has no operator-level fault tolerance. This is because state checkpoints are assigned to RDDs which is at the batch level

when a batch is completed (Vogel et al., 2024). Native continuous-operator systems like Apache Flink however, uses global consistent snapshots, created during failure-free time where when an operator fails, all operators can just rollback to the last saved checkpoint (Wang et al., 2022).

Vogel et al. (2024) made a benchmarking analysis testing the fault tolerance of three stream processing frameworks. The results suggests that Apache Flink is the most stable and has the best fault recovery as it has shorter downtime and fastest recovery. Apache Spark Streaming also provides a considerably fast and stable recovery and is an alternative if low latency is not a priority.

In terms of scalability, both frameworks scale linearly with resources. Apache Flink performs well even with heavy data loads and limited resources (Vyas & Kaushik, 2025). However, Apache Spark Streaming struggles with performance under high workloads. This is due to Apache Spark Streaming allocating the largest amount of memory space for processing compared to Flink (Vogel et al., 2024). Under heavy loads, sudden memory usage accompanied by large memory allocation can cause in-memory-based issues such as excessive garbage collection operations, out-of-memory issues and data loss (Matteussi et al., 2022).

V. Conclusion

In conclusion, this article compared Apache Spark Streaming and Apache Flink that focuses on their streaming models, architectural designs and performance characteristics. The analysis shows that each framework has various advantages depending on the application's requirements. Because of its true streaming approach, Apache Flink delivers excellent low-latency processing and advanced real-time capabilities, making it ideal for applications that require real-time responsiveness. On the other hand, Apache Spark Streaming has higher latency due to its micro-batch processing approach but still has high throughput, strong fault tolerance and seamless integration with the broader Spark ecosystem, making it a viable option for near real-time analytics.

Although it has limitations, Apache Spark Streaming is still an important and commonly used solution, especially in businesses who already use Apache Spark for batch processing and analytics. Future improvements in stream processing such as the transition to Structured Streaming and unified processing models are expected to further close the gap between batch and real-time analytics which allows for more efficient and adaptable data processing solutions.

References

[1] *Matteussi, K. J., dos Anjos, J. C. S., Leithardt, V. R. Q., & Geyer, C. F. R. (2022). Performance Evaluation Analysis of Spark Streaming Backpressure for Data-Intensive Pipelines. Sensors, 22(13), 4756.* https://doi.org/10.3390/s22134756

[2] *Ebada, A.I., Elhenawy, I., Jeong, C., Nam, Y., Elbakry, H. et al. (2022). Applying Apache Spark on Streaming Big Data for Health Status Prediction. Computers, Materials & Continua, 70(2), 3511–3527.* https://doi.org/10.32604/cmc.2022.019458

[3] *MEZATI, M., & AOURIA, I. (2025). Machine learning in big data: A performance benchmarking study of Flink-ML and Spark MLlib. Applied Computer Science, 21(2), 18–27.* https://doi.org/10.35784/acs_7297

[4] *Fedorovych, I., Osukhivska, H., & Lutsyk, N. (2024, November). Performance Benchmarking of Continuous Processing and Micro-Batch Modes in Spark Structured Streaming. In ITTAP (pp. 80-90).* https://ceur-ws.org/Vol-3896/paper5.pdf

[5] S. Vyas and S. Kaushik, "Efficient resource utilization in Flink stream processing with accuracy awareness and load shedding strategies," *6th International Conference on Recent Trends in Communication & Intelligent Systems (ICRTCIS 2025)*, Hybrid Conference, Jaipur, India, 2025, pp. 76-89, doi: https://doi.org/10.1049/icp.2025.2735

[6] Q. Le, M. Chen and W. Chen, "Research on stream processing engine and benchmarking framework," *2022 IEEE International Conference on Networking, Sensing and Control (ICNSC)*, Shanghai, China, 2022, pp. 1-5, doi: https://doi.org/10.1109/ICNSC55942.2022.10004188

[7] Daksa, R. P., & Kemala, A. P. (2025). A Comparative Study On Real Time Data Streaming For Fraud Detection Using Kafka With Apache Flink And Apache Spark. *Procedia Computer Science*, *269*, 192-199. https://doi-org.ezproxy.utm.my/10.1016/j.procs.2025.08.272

[8] Wang, X., Zhang, C., Fang, J. *et al.* A comprehensive study on fault tolerance in stream processing systems. *Front. Comput. Sci.* **16**, 162603 (2022). https://doi-org.ezproxy.utm.my/10.1007/s11704-020-0248-x

[9] Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., & Rabiser, R. (2024, June). A comprehensive benchmarking analysis of fault recovery in stream processing frameworks. In *PROCEEDINGS of the 18th ACM International Conference on Distributed and Event-based Systems* (pp. 171-182). https://doi.org/10.1145/3629104.3666040

UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3133-01

HIGH PERFORMANCE DATA PROCESSING

Assignment 1 Logbook

LECTURER: DR. MOHD SHAHIZAN BIN OTHMAN

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BIN MOHD NIZWAN	A23CS0084

Date	Search Keywords	AI Prompts Used	Task Description
6 April 2026	Apache Spark Streaming on WebOfScience (WoS)	None	Searched WoS for papers on Apache Spark Streaming. Found 5 relevant articles. Selected 2 to read.
10 April 2026	Apache Flink on WoS	None	Searched WoS for papers on Apache Spark Streaming. Found 5 relevant articles. Selected 3 to read.
12 April 2026	None	Find peer-reviewed research papers that compare stream processing frameworks such as Apache Spark Streaming and Apache Flink. Focus on studies that evaluate performance metrics such as latency, throughput, scalability, fault tolerance, and resource utilization in real-time big data processing systems. Prefer papers published after 2021 in journals or conferences related to big data, distributed systems, or data engineering using Scopus AI	Used Scopus AI to find papers that include comparison metrics on both Apache Spark Streaming and Apache Flink. Selected Found and 11 related articles.
14 April 2026	None	None	Created a draft comparison table comparing both frameworks in terms of streaming model, latency, throughput, fault tolerance and scalability.

17 April 2026	None	Give me a draft on the structure of the paper based on the assignment briefing file. Provide points I should include based on the topic given suing ChatGPT	Write introduction overview on Apache Spark Streaming.
18 April 2026	None	Explain to me on the Apache Spark Streaming architecture using Claude	Drew an architecture diagram using Lucid chart and explain it. Write on real-word case study on using Apache Spark Streaming for health status prediction.
19 April 2026	None	None	Create a comparison table and compare on streaming model, latency and throughput of both frameworks.
22 April 2026		Based on this paper, explain why micro-batch models like Apache Spark Streaming lacks operator-level fault tolerance while native-streaming model does not. Relate with the types of operators (stateful and stateless) and answer which model is more fault tolerant and why using Claude	Explain on comparison for fault tolerance and scalability.
25 April 2026	None	None	Write on the abstract, conclusion and keywords. Format and finalize the paper
26 April 2026	None	None	Submitted the paper on Turnitin. Made changes on flagged sentences
27 April 2026	None	None	Resubmitted the paper and finalize the final submission

ImanAbadi.pdf

by Iman Abadi MOHD NIZWAN

Submission date: 27-Apr-2026 12:27AM (UTC-0700)
Submission ID: 2944000627
File name: ImanAbadi.pdf (270.83K)
Word count: 2296
Character count: 13575

A Comparative Analysis of Apache Spark Streaming and Apache Flink for Real-Time Data Processing Performance

Iman Abadi Bin Mohd Nizwan
Faculty of Computing
University Technology Malaysia
Johor Bahru, Malaysia
imanabadi@graduate.utm.my

***Abstract*— The rapid growth of real-time data generated from applications such as Internet of Things (IoT), financial systems and online platforms. This has created a demand for efficient stream processing frameworks capable of delivering low-latency analytics. This paper focuses on Apache Spark Streaming as a widely used stream processing framework and examines its architecture, processing model and performance characteristics. This paper also explores on comparative analysis between Apache Spark Streaming against Apache Flink, with evaluation criteria including processing model, latency, throughput, scalability and fault tolerance. The findings shows that Apache Flink achieves better performance in terms of low-latency event-driven processing but Apache Spark Streaming remains highly effective for near real-time analytics due to its micro-batch model, strong fault tolerance and seamless integration within the Spark ecosystem.**

Keywords— Apache Spark Streaming, Apache Flink, Stream Processing; Real-Time Analytics

I. Introduction

The growth of real-time data from various types of applications and software has caused a need for an efficient real-time data processing solution. Applications and software such fraud detection systems, social media and Internet of Things (IoT) platforms produce high-velocity data that requires immediate insights and action. Traditional batch processing frameworks such as Apache Hadoop, although good for big data processing, lack the ability for low-latency data processing needed for real-time analysis. Hadoop is known to have high-latency and disk I/O overhead that significantly reduces performance time (Matteussi et al., 2022). To address these challenges, a stream processing framework offers better performance for stream analytics. Examples of streaming frameworks include Apache Spark Streaming, which uses a micro-batch model and Apache Flink, which uses a native streaming model. Both frameworks have their advantages and disadvantages in terms of performance, scalability and fault tolerance, for example. Selecting an appropriate streaming framework is crucial for an efficient streaming system that can optimize system performance and resource usage while meeting the requirements for a real-time processing system. This paper focuses on Apache Spark Streaming as a modern stream processing framework and compares it with Apache Flink based on key performance criteria, including processing model, latency, scalability and fault tolerance. The paper is structured into Apache Spark Streaming and architecture overview, a real-world case study, comparative analysis among stream processing frameworks and conclusion.

II. Apache Spark Streaming And Achitecture Overview

Apache Spark Streaming is a stream processing framework that offers rapid processing of large amounts of data through in-memory computations. This is preferrable compared to traditional map-reducing methods because it relies on disk operations, increasing I/O overhead, thus increasing latency. Spark Streaming processes data up to 100 times faster compared to Hadoop's map-reducing method (Ebada et al,. 2022).

Apache Spark Streaming uses micro-batching techniques, processing data in discrete intervals, and combining them to have continuous stream-like data which are then processed using Spark's underlying distributed computing engine which in turn enables it to have high throughput and fault tolerance (Fedorovych et al., 2024). The micro-batches are called Resilient Distributed Datasets (RDDs), a data abstraction method which are separate data blocks organized into small groups that corresponds to a batch data received at specific time interval which can be processed in parallel (Matteussi et al., 2022). The blocks are then sequenced overtime to form stream-like data. The architecture can be described in Fig. 1.

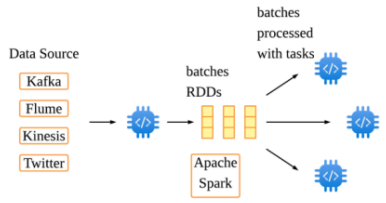


Fig 1. Apache Spark Streaming Architecture

Apache Spark Streaming also has excellent integration with other components in the Spark ecosystem such as MLlib. Apache Spark's MLlib is a Machine Learning (ML) library optimized for batch processing and micro-batch streaming (MEZATI & AOURIA, 2025). This integration provides a unified platform for complex analytics and machine learning which eliminates the need for multiple processing systems, simplifying the development of an end-to-end data pipeline.

Although Apache Spark Streaming is famous for its high throughput and scalability, it still lacks extremely low latency processing compared to other frameworks. In order to address this issue, Apache Spark Streaming introduced a more Structured Streaming which includes a new processing

model called Continuous Processing. This is done by using long-lived operators, keeping states across data batches. This reduces overhead that comes with micro-batch processing, having multiple task starts and stops (Fedorovych et al., 2024). This shows that Apache Spark Streaming gives option various application in stream analytics from using micro-batch processing for fault tolerance, high throughput and scalability or using Continuous Processing for improved latency for near real-time data processing.

III. Case Study: Apache Spark Streaming For Health Status Prediction

Ebada et al. (2022) proposed on using Apache Spark for streaming big data for health status prediction. The paper introduces an intelligent and scalable data architecture designed to monitor and predict health issues by leveraging on cloud computing and IoT devices. The primary goal is to create a system that interprets continuous medical data from wearable biosensors like heart rate, blood pressure, and body temperature trackers to detect health risks before they become critical. By combining past electronic health records (EHRs) with live streaming data, the system can deliver personalized suggestions and real-time alerts to patients and healthcare providers. Apache Spark is used for its capability to handle high-velocity healthcare data from IoT devices to extract medical data which will then be used by Apache Spark's machine learning library, MLlib to apply classification algorithms such as multiclass neural network and multiclass random forests. In summary, this study highlights the capabilities of Apache Spark Streaming in managing real-time medical data.

IV. Comparative Analysis on Stream Processing Frameworks

This section compares Apache Spark Streaming, a stream processing framework with another framework called Apache Flink. Apache Flink is an open-sourced framework that is known for distributed stream processing that is scalable, stateful and fault-tolerant (Vyas & Kaushik, 2025).

TABLE I. Comparison of Stream Processing Frameworks

Criteria	Apache Spark Streaming	Apache Flink
Processing Model	Micro-batch by default (Daksa & Kamala, 2025). Supports Continuous Streaming for true streaming (Fedorovych et al., 2024)	Native streaming (Daksa & Kamala, 2025)
Latency	Ranges from milliseconds up to seconds with more transactions (Le et al., 2022)	Milliseconds (Le et al., 2022)
Throughput	High, slows down under heavy load (Le et al., 2022)	High (Le et al., 2022)
Fault Tolerance	Checkpoint based on RDD (Le et al., 2022)	Checkpoint + Savepoint (Le et al., 2022)
Scalability	Linear with resources, presents in-memory issues under heavy load (Matteussi et al., 2022)	Linear with resources, better performing even under limited resources (Vyas & Kaushik, 2025)

A. Processing Model Comparison

The main difference between Apache Spark Streaming and Apache Flink is the processing models used which significantly effects system performance, latency and suitable use cases. Spark Streaming uses a micro-batching processing model where incoming data are separated into small batches and processed and s set time interval. Apache Flink on the other hand, is known for its true streaming approach, which processes individual events with sub-second latency and fine-grained state management (Daksa & Kamala, 2025).

Daksa & Kamala (2025) discussed that each paradigms has their own strength and specific use cases based on their literature review. Apache Spark Streaming's micro-batch approach ensures stronger fault tolerance but causes higher latency in term of processing speed. Flink on the other hand is more suited for real-time event processing as it offers low latency and higher processing consistency.

B. Performance Evaluation (Latency and Throughput)

A study by Le et al. (2022) showed that Apache Flink is better than Apache Spark Streaming in terms of performance in shuffle, stateful computation and windowed computation for both latency and throughput. The test was done using HiBench Streaming Benchmark using four types of test loads namely identity, repartition, stateful wordcount and fixwindow. In terms of throughput, results show that Apache Flink consistently outperforms Apache Spark Streaming. For identity and fixwindow test loads, both frameworks showed similar throughput up until 100,000 data records per second where Apache Sparks Streaming starts to drop. Apache Spark Streaming performed worse in repartition and wordcount test loads where throughput drops at 60,000 and 80,000 data records per second while Apache Flink maintains consistently high throughput. In terms of latency, Apache Flink consistently have latency in the millisecond range even at high data streams. Apache Spark Streaming on the other hand has latency in milliseconds range at low data streams but increases at higher workloads.

Another study by Daksa & Kamala (2025) showed contrasting results compared to the previous study. Apache Spark Streaming achieved lower latency for both ingestion rates (10 and 40 transactions pr seconds). Daksa & Kamala (2025) acknowledge that the findings differ from other literature as Apache Flink is always known to be better at low-latency processing. The paper found that Apache Spark Streaming's improved performance could be associated with internal optimization and consistent batch sizing that is aligned with workload characteristics. The article also suggests that Apache Flink is better suited for stateful processing and complicated event scenarios, which is demonstrated by Le et al. (2022), whereas Spark is recommended for predictable event structures and stable data rates.

C. Fault Tolerance and Scalability

Different stream processing models have different capabilities in terms of fault tolerance. According to Wang et al. (2022), a micro-batch processing model used by Apache Spark Streaming is near inherently fault-tolerant due to the underlying robust data structures and lineage information but has no operator-level fault tolerance. This is because state checkpoints are assigned to RDDs which is at the batch level

when a batch is completed (Vogel et al., 2024). Native continuous-operator systems like Apache Flink however, uses global consistent snapshots, created during failure-free time where when an operator fails, all operators can just rollback to the last saved checkpoint (Wang et al., 2022).

Vogel et al. (2024) made a benchmarking analysis testing the fault tolerance of three stream processing frameworks. The results suggests that Apache Flink is the most stable and has the best fault recovery as it has shorter downtime and fastest recovery. Apache Spark Streaming also provides a considerably fast and stable recovery and is an alternative if low latency is not a priority.

In terms of scalability, both frameworks scale linearly with resources. Apache Flink performs well even with heavy data loads and limited resources (Vyas & Kaushik, 2025). However, Apache Spark Streaming struggles with performance under high workloads. This is due to Apache Spark Streaming allocating the largest amount of memory space for processing compared to Flink (Vogel et al., 2024). Under heavy loads, sudden memory usage accompanied by large memory allocation can cause in-memory-based issues such as excessive garbage collection operations, out-of-memory issues and data loss (Matteussi et al., 2022).

V. Conclusion

In conclusion, this article compared Apache Spark Streaming and Apache Flink that focuses on their streaming models, architectural designs and performance characteristics. The analysis shows that each framework has various advantages depending on the application's requirements. Because of its true streaming approach, Apache Flink delivers excellent low-latency processing and advanced real-time capabilities, making it ideal for applications that require real-time responsiveness. On the other hand, Apache Spark Streaming has higher latency due to its micro-batch processing approach but still has high throughput, strong fault tolerance and seamless integration with the broader Spark ecosystem, making it a viable option for near real-time analytics.

Although it has limitations, Apache Spark Streaming is still an important and commonly used solution, especially in businesses who already use Apache Spark for batch processing and analytics. Future improvements in stream processing such as the transition to Structured Streaming and unified processing models are expected to further close the gap between batch and real-time analytics which allows for more efficient and adaptable data processing solutions.

References

[1] *Matteussi, K. J., dos Anjos, J. C. S., Leithardt, V. R. Q., & Geyer, C. F. R. (2022). Performance Evaluation Analysis of Spark Streaming Backpressure for Data-Intensive Pipelines. Sensors, 22(13), 4756. https://doi.org/10.3390/s22134756*

[2] *Ebada, A.I., Elhenawy, I., Jeong, C., Nam, Y., Elbakry, H. et al. (2022). Applying Apache Spark on Streaming Big Data for Health Status Prediction. Computers, Materials & Continua, 70(2), 3511–3527. https://doi.org/10.32604/cmc.2022.019458*

[3] *MEZATI, M., & AOURIA, I. (2025). Machine learning in big data: A performance benchmarking study of Flink-ML and Spark MLlib. Applied Computer Science, 21(2), 18–27. https://doi.org/10.35784/acs_7297*

[4] *Fedorovych, I., Osukhivska, H., & Lutsyk, N. (2024, November). Performance Benchmarking of Continuous Processing and Micro-Batch Modes in Spark Structured Streaming. In ITTAP (pp. 80-90). https://ceur-ws.org/Vol-3896/paper5.pdf*

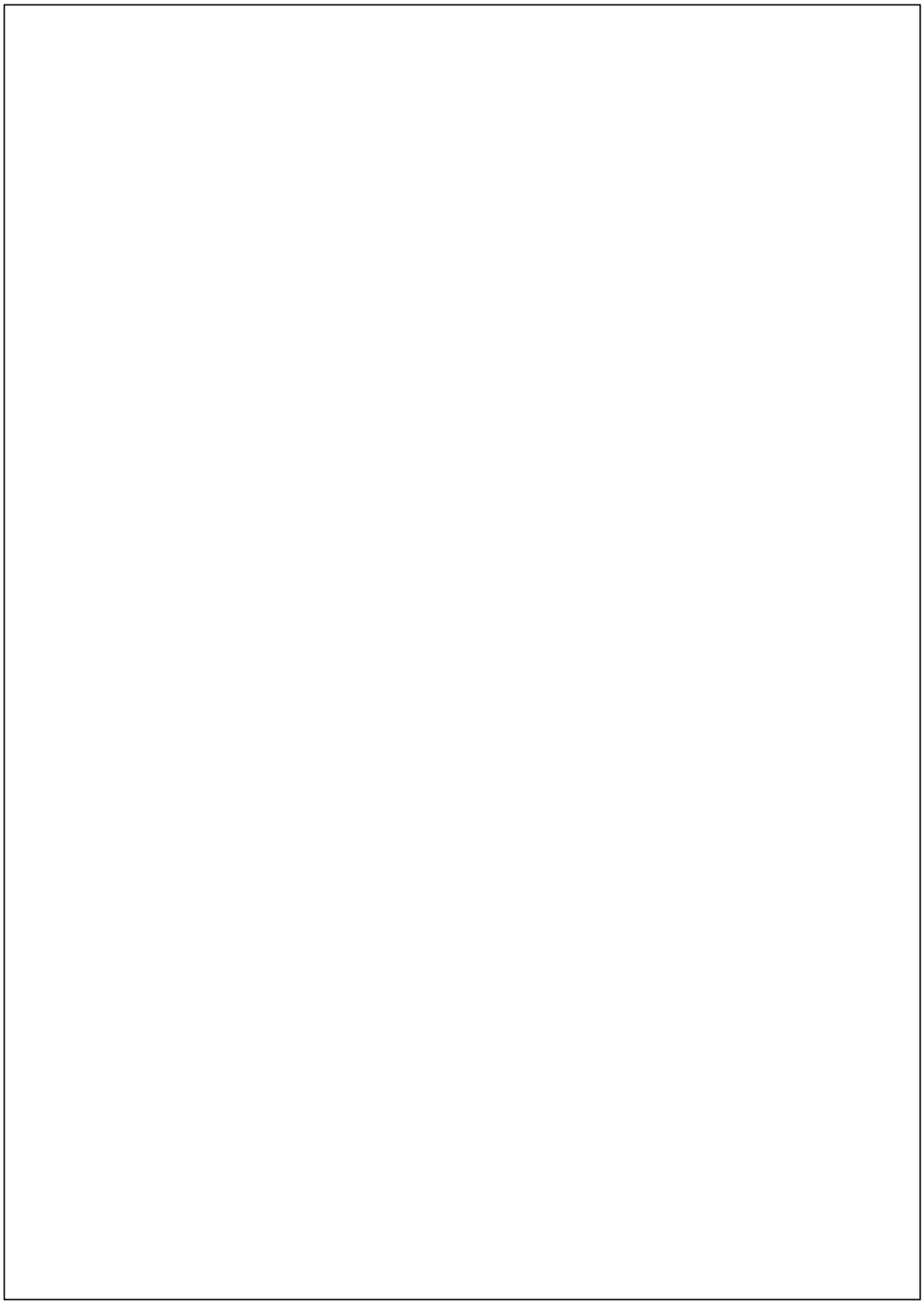
[5] S. Vyas and S. Kaushik, "Efficient resource utilization in Flink stream processing with accuracy awareness and load shedding strategies," *6th International Conference on Recent Trends in Communication & Intelligent Systems (ICRTCIS 2025)*, Hybrid Conference, Jaipur, India, 2025, pp. 76-89, doi: https://doi.org/10.1049/icp.2025.2735

[6] Q. Le, M. Chen and W. Chen, "Research on stream processing engine and benchmarking framework," *2022 IEEE International Conference on Networking, Sensing and Control (ICNSC)*, Shanghai, China, 2022, pp. 1-5, doi: https://doi.org/10.1109/ICNSC55942.2022.10004188

[7] Daksa, R. P., & Kemala, A. P. (2025). A Comparative Study On Real Time Data Streaming For Fraud Detection Using Kafka With Apache Flink And Apache Spark. *Procedia Computer Science*, *269*, 192-199. https://doi-org.ezproxy.utm.my/10.1016/j.procs.2025.08.272

[8] Wang, X., Zhang, C., Fang, J. *et al*. A comprehensive study on fault tolerance in stream processing systems. *Front. Comput. Sci.* **16**, 162603 (2022). https://doi-org.ezproxy.utm.my/10.1007/s11704-020-0248-x

[9] Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., & Rabiser, R. (2024, June). A comprehensive benchmarking analysis of fault recovery in stream processing frameworks. In *PROCEEDINGS of the 18th ACM International Conference on Distributed and Event-based Systems* (pp. 171-182). https://doi.org/10.1145/3629104.3666040



ImanAbadi.pdf

ORIGINALITY REPORT

8%	3%	6%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Qionghua Le, Mingang Chen, Wenjie Chen. "Research on stream processing engine and benchmarking framework", 2022 IEEE International Conference on Networking, Sensing and Control (ICNSC), 2022 Publication	1%
2	Xiaotong Wang, Chunxi Zhang, Junhua Fang, Rong Zhang, Weining Qian, Aoying Zhou. "A comprehensive study on fault tolerance in stream processing systems", Frontiers of Computer Science, 2021 Publication	1%
3	Submitted to Peirce College Student Paper	1%
4	Rayhan Prawira Daksa, Ade Putera Kemala. "A Comparative Study On Real Time Data Streaming For Fraud Detection Using Kafka With Apache Flink And Apache Spark", Procedia Computer Science, 2025 Publication	1%
5	Submitted to Universiti Teknologi Malaysia Student Paper	1%
6	Submitted to University of West London Student Paper	1%
7	labrys.io Internet Source	1%
8	Max Petrov, Nikolay Butakov, Denis Nasonov, Mikhail Melnik. "Adaptive performance model	1%

	for dynamic scaling Apache Spark Streaming", Procedia Computer Science, 2018 Publication	
9	ph.pollub.pl Internet Source	1 %
10	www.mdpi.com Internet Source	1 %
11	ceur-ws.org Internet Source	<1 %
12	Messaoud MEZATI, Ines AOURIA. "Machine learning in big data: A performance benchmarking study of Flink-ML and Spark MLlib", Applied Computer Science, 2025 Publication	<1 %

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		